SISTEM MANAJEMEN DATABASE TERINTEGRASI UNTUK DINOYO CAMERA

SEBAGAI LANGKAH MENDUKUNG TEKNOLOGI UMKM DI INDONESIA DENGAN DIGITALISASI

PENCIPTA

Aan Anjar Setyowati (240535603752)
Muhammad Farel Alfathir (240535607790)
Qurrotul Aini (240535603434)
Abdullah Ali Wafa (240535606879)
Shafwan Adhi Dwi Nugraha (240535608339)
Dea Alensa (230533607297)
Ananda Dhiaz Nikola (230533602149)
Dr. Ir. Triyanna Widiyaningtyas, M.T
Heni Vidia Sari, S.P., M. Kom.
Dyah Lestari, S.T., M.Eng.
Syaiful Budiyanto

I. Judul

Sistem Manajemen Database Terintegrasi untuk Dinoyo Camera sebagai Langkah Mendukung Teknologi UMKM di Indonesia dengan Digitalisasi

II. Latar Belakang

Usaha Mikro, Kecil, dan Menengah (UMKM) merupakan tulang punggung perekonomian Indonesia. Menurut data Kementerian Koperasi dan UKM, lebih dari 99% pelaku usaha di Indonesia berasal dari sektor UMKM, dengan kontribusi signifikan terhadap Produk Domestik Bruto (PDB) serta penyerapan tenaga kerja. Namun, di tengah pesatnya perkembangan teknologi informasi, sebagian besar UMKM masih menghadapi tantangan besar dalam melakukan transformasi digital, khususnya dalam hal manajemen data dan sistem informasi yang terintegrasi.

Digitalisasi menjadi kebutuhan mendesak bagi UMKM agar mampu bersaing, beradaptasi, dan berkembang di era ekonomi digital. Salah satu aspek fundamental dari digitalisasi adalah pengelolaan data yang efisien, akurat, dan terstruktur. Penggunaan sistem basis data relasional dapat menjadi solusi penting dalam meningkatkan efisiensi operasional, transparansi transaksi, dan pengambilan keputusan berbasis data.

Melalui tugas akhir mata kuliah Basis Data pada Program Studi Teknik Informatika Universitas Negeri Malang, mahasiswa diberikan kesempatan untuk menerapkan konsep-konsep yang telah dipelajari dalam dunia nyata melalui kerja sama dengan mitra usaha. Dalam konteks ini, tim mahasiswa menjalin kolaborasi dengan Dinoyo Camera, sebuah toko kamera yang termasuk dalam kategori UMKM. Dinoyo Cam bergerak di bidang penjualan dan penyewaan kamera serta perlengkapan fotografi, dan masih menghadapi tantangan dalam manajemen data transaksi serta inventaris produknya.

Sebagai bentuk kontribusi nyata terhadap transformasi digital UMKM, dikembangkanlah sebuah sistem basis data relasional yang dirancang khusus untuk mendukung kebutuhan operasional Dinoyo Cam. Sistem ini meliputi pengelolaan data produk, transaksi penjualan dan pembelian, informasi pelanggan, manajemen karyawan, serta data cabang perusahaan. Pengembangan sistem basis data ini menjadi langkah awal dalam membangun fondasi sistem informasi yang lebih luas dan terintegrasi bagi UMKM.

Dokumentasi dari sistem ini disusun dalam bentuk manual book berjudul *"Manajemen Database Terintegrasi untuk Dinoyo Camera sebagai Langkah Mendukung Teknologi UMKM di Indonesia dengan Digitalisasi"*. Manual ini secara khusus berfokus pada aspek basis data relasional sebagai inti dari sistem yang dibangun, bukan pada aplikasi web-nya secara keseluruhan. Hal ini karena tujuan utamanya adalah sebagai bagian dari tugas akhir yang akan diajukan untuk mendapatkan Hak Kekayaan Intelektual (HKI).

Basis data yang dikembangkan diberi nama *db_dinoyokamera*, dan terdiri atas 13 tabel utama yang mencakup seluruh kebutuhan operasional toko, yaitu: *users*, *produk*, *penjualan*, *pembelian*, *kondisi*, *kategori*, *karyawan*, *perusahaan_cabang*, *detail_produk*, *detail_penjualan*, *detail_pembelian*, dan *customer*. Relasi antar tabel dibangun dengan prinsip normalisasi dan integritas data agar mendukung performa dan skalabilitas sistem dalam jangka panjang.

Melalui penyusunan manual book ini, diharapkan pihak Dinoyo Camera sebagai pengguna sistem dapat memahami struktur dan alur kerja basis data yang digunakan. Selain itu, proyek ini juga menjadi salah satu kontribusi mahasiswa dalam mendukung digitalisasi UMKM di Indonesia, dengan menghadirkan solusi teknologi yang praktis, aplikatif, dan dapat dikembangkan lebih lanjut sesuai kebutuhan. Dengan demikian, kegiatan ini tidak hanya memberikan manfaat langsung kepada mitra, tetapi juga memperkuat sinergi antara dunia pendidikan dan dunia usaha dalam mewujudkan transformasi digital nasional.

III. Tujuan

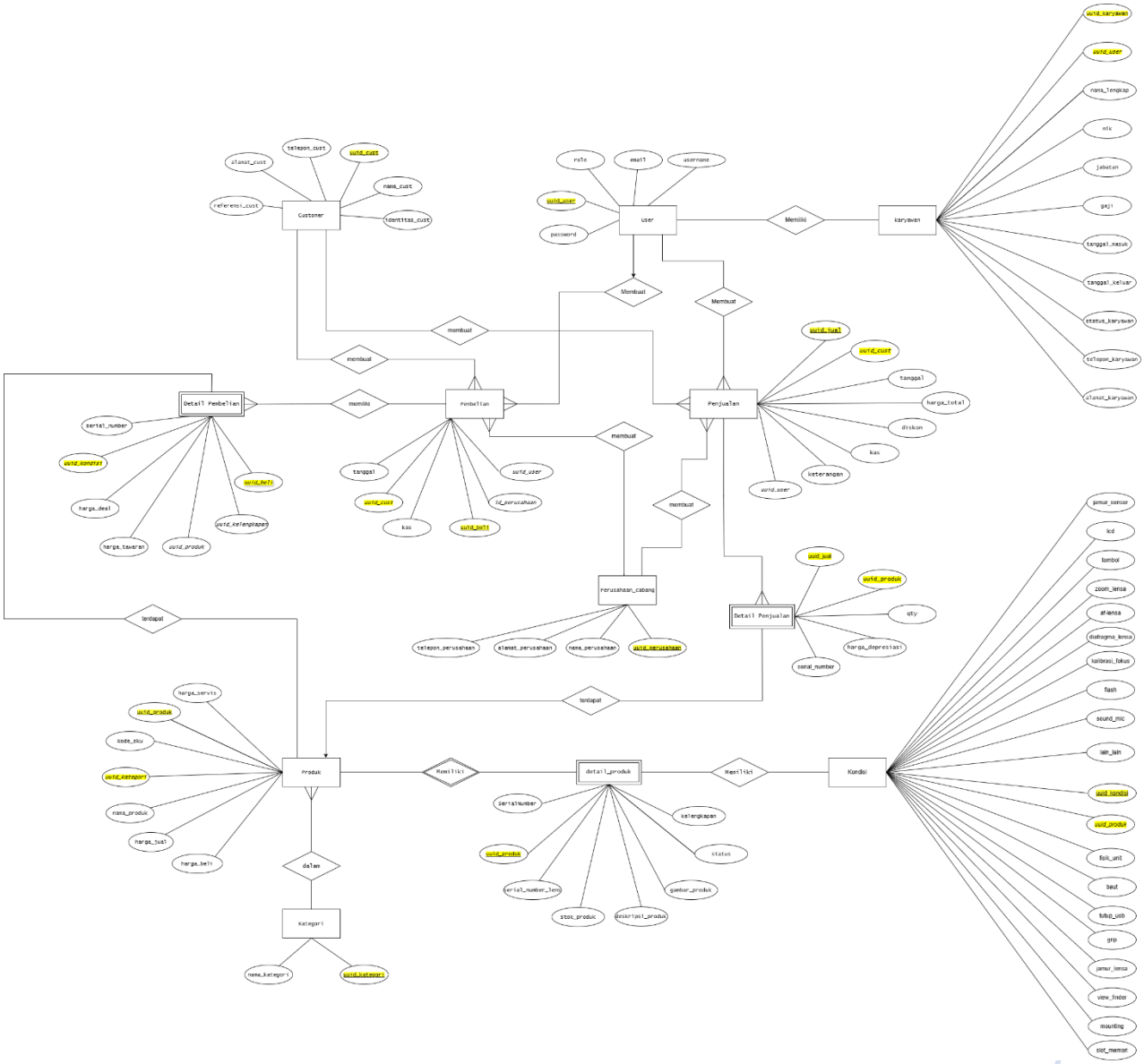
Penyusunan *manual book* ini bertujuan untuk:

a) Mengimplementasikan konsep basis data relasional dalam sistem nyata berbasis kebutuhan mitra UMKM, yaitu Dinoyo Camera.
b) Menyediakan dokumentasi teknis sebagai panduan penggunaan dan pemahaman sistem *database* yang telah dikembangkan.
c) Mendukung proses digitalisasi usaha kecil melalui pengelolaan data yang terstruktur dan efisien.
d) Membangun fondasi sistem informasi yang fleksibel dan dapat dikembangkan di masa depan.

e) Menjadi bentuk kontribusi mahasiswa dalam mendukung transformasi digital UMKM di Indonesia.

f) Memenuhi syarat administrasi untuk pengajuan Hak Kekayaan Intelektual (HKI) atas hasil karya sistem *database* ini.

IV. Rancangan *Database* yang Dikembangkan



Gambar 1.1 Rancangan Basis Data

V. *Source Code*

a. *Database* db_dinoyokamera

```
CREATE DATABASE db_dinoyokamera;
```

Query ini digunakan untuk **membuat sebuah basis data baru** yang diberi nama `db_dinoyokamera`. Basis data ini akan menjadi tempat menyimpan semua tabel dan prosedur terkait sistem.

b. Tabel users

```
CREATE TABLE users (
  uuid_user INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(20) NOT NULL,
  password VARCHAR(20) NOT NULL,
  email VARCHAR(20) NOT NULL,
  role ENUM('admin', 'kasir', 'staff') NOT NULL
);
```

Tabel ini digunakan untuk menyimpan data seluruh pengguna yang memiliki akses ke sistem, termasuk *username*, *password*, email, dan perannya.

c. Tabel produk

```
CREATE TABLE produk (
   uuid_produk VARCHAR(36) PRIMARY KEY NOT NULL,
   kode_sku VARCHAR(20) NOT NULL,
   uuid_kategori VARCHAR(36) NOT NULL,
   nama_produk VARCHAR(50) NOT NULL,
   harga_jual INT DEFAULT NULL,
   harga_beli INT DEFAULT NULL,
   harga_servis INT DEFAULT NULL,
   FOREIGN KEY (uuid_kategori) REFERENCES kategori(uuid_kategori)
);
```

Tabel ini digunakan untuk menyimpan informasi utama tentang produk yang tersedia, seperti nama, harga, kategori, dan kode SKU.

d. Tabel penjualan

```
CREATE TABLE penjualan (
   uuid_jual VARCHAR(36) PRIMARY KEY NOT NULL,
   uuid_cust VARCHAR(36) NOT NULL,
   uuid_perusahaan VARCHAR(36) NOT NULL,
   uuid_user VARCHAR(36) NOT NULL,
   harga_total INT NOT NULL,
   diskon DOUBLE,
   kas ENUM('transfer', 'cash') NOT NULL,
   keterangan VARCHAR(200),
```

```
    tanggal DATE NOT NULL,
    FOREIGN KEY (uuid_cust) REFERENCES customer(uuid_cust),
    FOREIGN KEY (uuid_perusahaan)
     REFERENCES perusahaan_cabang(uuid_perusahaan),
    FOREIGN KEY (uuid_user) REFERENCES users(uuid_user)
);
```

Tabel ini digunakan untuk menyimpan catatan transaksi pembelian dari pelanggan ke perusahaan, termasuk tanggal pembelian, metode pembayaran, dan cabang yang terkait.

e. Tabel pembelian

```
CREATE TABLE pembelian (
    uuid_beli VARCHAR(36) PRIMARY KEY NOT NULL,
    uuid_cust VARCHAR(36) NOT NULL,
    uuid_perusahaan VARCHAR(36) NOT NULL,
    uuid_user VARCHAR(36) NOT NULL,
    tanggal DATE NOT NULL,
    kas ENUM('transfer', 'cash'),
    FOREIGN KEY (uuid_cust) REFERENCES customer(uuid_cust),
    FOREIGN KEY (uuid_perusahaan)
    REFERENCES perusahaan_cabang(uuid_perusahaan),
    FOREIGN KEY (uuid_user) REFERENCES users(uuid_user)
);
```

Tabel ini digunakan untuk menyimpan catatan transaksi pembelian dari pelanggan ke perusahaan, termasuk tanggal pembelian, metode pembayaran, dan cabang yang terkait.

f. Tabel kondisi

```
CREATE TABLE kondisi (
   uuid_kondisi VARCHAR(36) PRIMARY KEY NOT NULL,
   fisik VARCHAR(100),
   baut VARCHAR(50),
   tutup_usb VARCHAR(50),
   grip VARCHAR(50),
   jamur_lensa VARCHAR(100),
   view_finder VARCHAR(50),
   mounting VARCHAR(50),
   slot_memori VARCHAR(50),
   jamur_sensor VARCHAR(100),
   lcd VARCHAR(100),
   tombol VARCHAR(50),
   zoom_lensa VARCHAR(50),
   af_lensa VARCHAR(50),
   diafragma_lensa VARCHAR(50),
   kalibrasi_fokus VARCHAR(50),
   flash VARCHAR(100),
   sound_mic VARCHAR(50),
   lain_lain VARCHAR(255)
```

```
);
```

Tabel ini digunakan untuk menyimpan detail kondisi fisik barang bekas yang dibeli atau dijual, seperti kondisi fisik, lensa, tombol, selot memori, dan lain-lain.

g. Tabel kategori

```
CREATE TABLE kategori (
   uuid_kategori VARCHAR(36) PRIMARY KEY NOT NULL,
   nama_kategori ENUM('Kamera','Lensa','Aksesoris') NOT NULL
);
```

Tabel ini digunakan untuk menyimpan daftar kategori produk, misalnya: Kamera, Lensa, atau Aksesoris. Kategori ini digunakan untuk mengelompokkan produk.

h. Tabel karyawan

```
CREATE TABLE karyawan (
   uuid_karyawan INT AUTO_INCREMENT PRIMARY KEY,
   uuid_user INT NOT NULL,
   nama_lengkap VARCHAR(50) NOT NULL,
   nik VARCHAR(20) NOT NULL,
   jabatan VARCHAR(50) NOT NULL,
   gaji INT NOT NULL,
   tanggal_masuk DATE NOT NULL,
   tanggal_keluar DATE,
   status ENUM('aktif', 'non-aktif') NOT NULL,
   nomor_telepon VARCHAR(15) NOT NULL,
   alamat VARCHAR(100) NOT NULL,
   FOREIGN KEY (uuid_user) REFERENCES users(uuid_user)
);
```

Tabel ini digunakan untuk menyimpan data seluruh karyawan, termasuk data personal, jabatan, gaji, serta status kerja mereka (aktif atau nonaktif).

i. Tabel perusahaan_cabang

```
 CREATE TABLE perusahaan_cabang (
   uuid_perusahaan VARCHAR(36) PRIMARY KEY NOT NULL,
   nama VARCHAR(50) NOT NULL,
   alamat VARCHAR(100) NOT NULL,
   nomor_telepon VARCHAR(20) NOT NULL
 );
```

Tabel ini digunakan untuk menyimpan data mengenai cabang-cabang perusahaan, termasuk nama, alamat, dan nomor telepon.

j. Tabel detail_produk

```sql
CREATE TABLE detail_produk (
  uuid_produk VARCHAR(36) PRIMARY KEY NOT NULL,
  serial_number VARCHAR(20),
  serial_number_lens VARCHAR(20),
  stok_produk INT NOT NULL,
  deskripsi_produk TEXT,
  gambar_produk BLOB,
  status_produk ENUM('baru','second') DEFAULT NULL,
  kelengkapan TEXT,
  FOREIGN KEY (uuid_produk) REFERENCES produk(uuid_produk)
);
```

Tabel ini digunakan untuk menyimpan informasi tambahan dari setiap produk, seperti nomor seri, stok, status produk (baru/second), dan deskripsi produk.

k. Tabel detail_penjualan

```sql
CREATE TABLE detail_penjualan (
  uuid_jual VARCHAR(36) NOT NULL,
  uuid_produk VARCHAR(36) NOT NULL,
  serial_number VARCHAR(30),
  qty INT NOT NULL,
  harga_depresiasi DOUBLE,
  PRIMARY KEY (uuid_jual, uuid_produk),
  FOREIGN KEY (uuid_jual) REFERENCES penjualan(uuid_jual),
  FOREIGN KEY (uuid_produk) REFERENCES produk(uuid_produk)
);
```

Tabel ini digunakan untuk menyimpan informasi lebih detail dari transaksi penjualan, seperti jumlah produk yang terjual dan harga setelah depresiasi.

l. Tabel detail_pembelian

```sql
CREATE TABLE detail_pembelian (
   uuid_beli VARCHAR(36) NOT NULL,
   uuid_produk VARCHAR(36) NOT NULL,
   serial_number VARCHAR(20),
   uuid_kondisi VARCHAR(36) NOT NULL,
   harga_tawaran INT,
   harga_terakhir INT,
   status_pembelian ENUM('deal','tidak'),
   PRIMARY KEY (uuid_beli, uuid_produk),
   FOREIGN KEY (uuid_beli) REFERENCES pembelian(uuid_beli),
   FOREIGN KEY (uuid_produk) REFERENCES produk(uuid_produk),
   FOREIGN KEY (uuid_kondisi) REFERENCES kondisi(uuid_kondisi)
);
```

Tabel ini digunakan untuk menyimpan informasi lebih detail dari setiap transaksi pembelian, seperti produk yang dibeli, harga tawaran, harga akhir, dan status negosiasi (deal atau tidak).

m. Tabel customer

```
CREATE TABLE customer (
   uuid_cust INT AUTO_INCREMENT PRIMARY KEY,
   nama VARCHAR(50) NOT NULL,
   identitas VARCHAR(20) NOT NULL,
   no_telp VARCHAR(20) NOT NULL,
   alamat VARCHAR(100) NOT NULL,
   jenis_kelamin ENUM('Laki-laki', 'Perempuan') NOT NULL,
   referensi VARCHAR(50),
   keterangan TEXT
);
```

Tabel ini digunakan untuk menyimpan informasi pelanggan yang melakukan transaksi, seperti nama, identitas, nomor telepon, alamat, jenis kelamin, dan catatan tambahan.

n. Stored Procedure create_user

```
DELIMITER $$
CREATE PROCEDURE create_user(
  IN p_username VARCHAR(20),
  IN p_password VARCHAR(255),
  IN p_email VARCHAR(50),
  IN p_role ENUM('admin','kasir','staff')
)
BEGIN
  INSERT INTO users (uuid_user, username, password, email, role)
  VALUES (UUID(), p_username, p_password, p_email, p_role);
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk **menambahkan pengguna baru** ke dalam sistem. Prosedur ini menerima informasi seperti username, password, email, dan peran pengguna (admin, kasir, atau staf).

o. Stored Procedure get_users

```
DELIMITER $$
CREATE PROCEDURE get_users()
BEGIN
  SELECT * FROM users;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk **mengambil seluruh data pengguna** dari tabel `users`.

p. **Stored Procedure update_user**

```
DELIMITER $$
CREATE PROCEDURE update_user(
  IN p_uuid_user VARCHAR(36),
  IN p_username VARCHAR(255),
  IN p_password VARCHAR(20),
  IN p_email VARCHAR(50),
  IN p_role ENUM('admin','kasir','staff')
)
BEGIN
  UPDATE users
  SET username = p_username,
      password = p_password,
      email = p_email,
      role = p_role
  WHERE uuid_user = p_uuid_user;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk **memperbarui data pengguna** yang sudah ada, seperti nama pengguna, kata sandi, email, dan peran, berdasarkan ID unik pengguna.

q. Stored Procedure delete_user

```
DELIMITER $$
CREATE PROCEDURE delete_user(IN p_uuid_user VARCHAR(36))
BEGIN
  DELETE FROM users WHERE uuid_user = p_uuid_user;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk **menghapus data pengguna** berdasarkan ID unik pengguna (`uuid_user`).

r. Stored Procedure create_karyawan

```
DELIMITER $$
CREATE PROCEDURE create_karyawan(IN p_username VARCHAR(20),
IN p_nama_lengkap VARCHAR(50), IN p_nik VARCHAR(20),
IN p_jabatan VARCHAR(50), IN p_gaji INT, IN p_tanggal_masuk DATE,
IN p_tanggal_keluar DATE, IN p_status ENUM('aktif','non-aktif'),
IN p_nomor_telepon VARCHAR(15), IN p_alamat VARCHAR(100))
BEGIN
  DECLARE v_uuid_user VARCHAR(36);
```

```
  DECLARE v_exists INT;

  SELECT uuid_user INTO v_uuid_user FROM users
      WHERE username = p_username LIMIT 1;

  IF v_uuid_user IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Username tidak
      ditemukan di tabel users';
  ELSE
    INSERT INTO karyawan (
      uuid_karyawan, uuid_user, nama_lengkap, nik, jabatan, gaji,
      tanggal_masuk, tanggal_keluar, status, nomor_telepon, alamat
    ) VALUES (
      UUID(), v_uuid_user, p_nama_lengkap, p_nik, p_jabatan, p_gaji,
      p_tanggal_masuk, p_tanggal_keluar, p_status, p_nomor_telepon,
       p_alamat
    );
  END IF;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk **menambahkan data karyawan baru** ke tabel karyawan. Sebelum menambahkan, sistem akan mencari *username* yang sesuai di tabel users. Jika tidak ditemukan, maka akan muncul pesan kesalahan.

s. Stored Procedure create_perusahaan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE create_perusahaan
(IN p_nama VARCHAR(50), IN p_alamat VARCHAR(100),
IN p_telp VARCHAR(20))
BEGIN
INSERT INTO perusahaan_cabang(uuid_perusahaan, nama, alamat,
nomor_telepon)
    VALUES (UUID(), p_nama, p_alamat, p_telp);
end$$
DELIMITER ;
```

Prosedur ini berfungsi untuk menambahkan data cabang perusahaan baru ke tabel perusahaan_cabang.

t. Stored Procedure delete_karyawan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE delete_karyawan(IN
p_uuid_karyawan VARCHAR(36))
BEGIN
    DELETE FROM karyawan WHERE uuid_karyawan = p_uuid_karyawan;
END$$
```

```
DELIMITER ;
```

Digunakan untuk menghapus data karyawan berdasarkan `uuid_karyawan` (ID unik dari karyawan).

u. Stored Procedure delete_perusahaan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE delete_perusahaan(IN
p_uuid_perusahaan VARCHAR(36))
BEGIN
DELETE FROM perusahaan_cabang WHERE uuid_perusahaan = p_uuid_perusahaan;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk menghapus data perusahaan (cabang) berdasarkan `uuid_perusahaan`.

v. Stored Procedure get_karyawan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE get_karyawan()
BEGIN
 SELECT * FROM karyawan;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan seluruh data yang ada dalam tabel `karyawan`.

w. Stored Procedure get_perusahaan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE get_perusahaan()
BEGIN
SELECT * FROM perusahaan_cabang;
end$$
DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan semua data dari tabel `perusahaan_cabang`.

x. Stored Procedure update_karyawan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE update_karyawan(
  IN p_uuid_karyawan VARCHAR(36),
  IN p_username VARCHAR(20),
  IN p_nama_lengkap VARCHAR(50),
```

```
  IN p_nik VARCHAR(20),
  IN p_jabatan VARCHAR(50),
  IN p_gaji INT,
  IN p_tanggal_masuk DATE,
  IN p_tanggal_keluar DATE,
  IN p_status ENUM('aktif','non-aktif'),
  IN p_nomor_telepon VARCHAR(15),
  IN p_alamat VARCHAR(100)
)
BEGIN
  DECLARE v_uuid_user VARCHAR(36);

  SELECT uuid_user INTO v_uuid_user
  FROM users
  WHERE username = p_username
  LIMIT 1;

  IF v_uuid_user IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Username tidak ditemukan di tabel users';
  END IF;

  UPDATE karyawan
  SET
    uuid_user = v_uuid_user,
    nama_lengkap = p_nama_lengkap,
    nik = p_nik,
    jabatan = p_jabatan,
    gaji = p_gaji,
    tanggal_masuk = p_tanggal_masuk,
    tanggal_keluar = p_tanggal_keluar,
    status = p_status,
    nomor_telepon = p_nomor_telepon,
    alamat = p_alamat
  WHERE uuid_karyawan = p_uuid_karyawan;
END$$
DELIMITER ;
```

Prosedur ini berfungsi untuk memperbarui data karyawan berdasarkan UUID. Sebelumnya, akan dilakukan validasi bahwa `username` yang diberikan benar-benar terdaftar di tabel `users`.

y. Stored Procedure update_perusahaan

```
DELIMITER $$
CREATE DEFINER=root@localhost PROCEDURE update_perusahaan(IN p_uuid_perusahaan VARCHAR(36), IN p_nama VARCHAR(50), IN p_alamat VARCHAR(100), IN p_telp VARCHAR(20))
BEGIN
```

```
    UPDATE perusahaan_cabang
    SET nama = p_nama,
    alamat = p_alamat,
    nomor_telepon = p_telp
    WHERE uuid_perusahaan = p_uuid_perusahaan;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk memperbarui informasi perusahaan cabang berdasarkan ID perusahaan (`uuid_perusahaan`).

z. Stored Procedure create_kategori

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `create_kategori` (IN
`p_nama_kategori` ENUM('Kamera','Lensa','Aksesoris'))
BEGIN
    INSERT INTO kategori (uuid_kategori, nama_kategori)
    VALUES (UUID(), p_nama_kategori);
END$$
DELIMITER ;
```

Prosedur ini berfungsi untuk menambahkan data kategori baru ke table kategori.

aa. Stored Procedure get_kategori

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `get_kategori` ()
BEGIN
    SELECT * FROM kategori;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan semua data dari tabel `kategori`.

bb. Stored Procedure delete_kategori

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `delete_kategori` (IN
`p_uuid_kategori` VARCHAR(36))
BEGIN
    DELETE FROM kategori
    WHERE uuid_kategori = p_uuid_kategori;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk menghapus data kategori berdasarkan `uuid_kategori`.

cc. Stored Procedure update_kategori

```
DELIMITER $$
CREATE DEFINER=`root`@`localhost` PROCEDURE `update_kategori` (IN
`p_uuid_kategori` VARCHAR(36), IN `p_nama_kategori`
ENUM('Kamera','Lensa','Aksesoris'))
BEGIN
    UPDATE kategori
    SET nama_kategori = p_nama_kategori
    WHERE uuid_kategori = p_uuid_kategori;
END$$
DELIMITER ;
```

Prosedur ini digunakan untuk memperbarui informasi kategori berdasarkan ID kategori (`uuid_kategori`).

dd. Stored Procedure create_produk

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `create_produk` (
  IN `p_uuid_produk` VARCHAR(36),
  IN `p_kode_sku` VARCHAR(20),
  IN `p_uuid_kategori` VARCHAR(36),
  IN `p_nama_produk` VARCHAR(50),
  IN `p_harga_jual` INT,
  IN `p_harga_beli` INT,
  IN `p_harga_servis` INT,
  IN `p_uuid_kondisi` VARCHAR(36),
  IN `p_fisik` VARCHAR(100),
  IN `p_baut` VARCHAR(50),
  IN `p_tutup_usb` VARCHAR(50),
  IN `p_grip` VARCHAR(50),
  IN `p_jamur_lensa` VARCHAR(100),
  IN `p_view_finder` VARCHAR(50),
  IN `p_mounting` VARCHAR(50),
  IN `p_slot_memori` VARCHAR(50),
  IN `p_jamur_sensor` VARCHAR(100),
  IN `p_lcd` VARCHAR(100),
  IN `p_tombol` VARCHAR(50),
  IN `p_zoom_lensa` VARCHAR(50),
  IN `p_af_lensa` VARCHAR(50),
  IN `p_diafragma_lensa` VARCHAR(50),
  IN `p_kalibrasi_fokus` VARCHAR(50),
  IN `p_flash` VARCHAR(100),
  IN `p_sound_mic` VARCHAR(50),
  IN `p_lain_lain` VARCHAR(255),
  IN `p_serial_number` VARCHAR(20),
  IN `p_serial_number_lens` VARCHAR(20),
  IN `p_stok_produk` INT,
```

```
  IN `p_deskripsi_produk` TEXT,
  IN `p_gambar_produk` BLOB,
  IN `p_status_produk` ENUM('baru','second'),
  IN `p_kelengkapan` TEXT
)
BEGIN
  DECLARE v_sku_exists INT;
  DECLARE v_kategori_exists INT;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;

  INSERT INTO produk VALUES (
    p_uuid_produk, p_kode_sku, p_uuid_kategori, p_nama_produk,
    p_harga_jual, p_harga_beli, p_harga_servis
  );

  INSERT INTO kondisi VALUES (
    p_uuid_kondisi, p_fisik, p_baut, p_tutup_usb, p_grip,
    p_jamur_lensa, p_view_finder, p_mounting, p_slot_memori,
p_jamur_sensor,
    p_lcd, p_tombol, p_zoom_lensa, p_af_lensa, p_diafragma_lensa,
    p_kalibrasi_fokus, p_flash, p_sound_mic, p_lain_lain
  );

  INSERT INTO detail_produk VALUES (
    p_uuid_produk, p_uuid_kondisi, p_serial_number,
p_serial_number_lens,
    p_stok_produk, p_deskripsi_produk, p_gambar_produk,
    p_status_produk, p_kelengkapan
  );

  COMMIT;
END$$

DELIMITER ;
```

Prosedur ini berfungsi untuk menambahkan data produk baru ke table produk.

ee. Stored Procedure update_produk

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `update_produk` (
  IN `p_uuid_produk` VARCHAR(36),
```

```
  IN `p_kode_sku` VARCHAR(20),
  IN `p_uuid_kategori` VARCHAR(36),
  IN `p_nama_produk` VARCHAR(50),
  IN `p_harga_jual` INT,
  IN `p_harga_beli` INT,
  IN `p_harga_servis` INT,
  IN `p_uuid_kondisi` VARCHAR(36),
  IN `p_fisik` VARCHAR(100),
  IN `p_baut` VARCHAR(50),
  IN `p_tutup_usb` VARCHAR(50),
  IN `p_grip` VARCHAR(50),
  IN `p_jamur_lensa` VARCHAR(100),
  IN `p_view_finder` VARCHAR(50),
  IN `p_mounting` VARCHAR(50),
  IN `p_slot_memori` VARCHAR(50),
  IN `p_jamur_sensor` VARCHAR(100),
  IN `p_lcd` VARCHAR(100),
  IN `p_tombol` VARCHAR(50),
  IN `p_zoom_lensa` VARCHAR(50),
  IN `p_af_lensa` VARCHAR(50),
  IN `p_diafragma_lensa` VARCHAR(50),
  IN `p_kalibrasi_fokus` VARCHAR(50),
  IN `p_flash` VARCHAR(100),
  IN `p_sound_mic` VARCHAR(50),
  IN `p_lain_lain` VARCHAR(255),
  IN `p_serial_number` VARCHAR(20),
  IN `p_serial_number_lens` VARCHAR(20),
  IN `p_stok_produk` INT,
  IN `p_deskripsi_produk` TEXT,
  IN `p_gambar_produk` BLOB,
  IN `p_status_produk` ENUM('baru','second'),
  IN `p_kelengkapan` TEXT
)
BEGIN
  UPDATE produk
  SET kode_sku = p_kode_sku,
      uuid_kategori = p_uuid_kategori,
      nama_produk = p_nama_produk,
      harga_jual = p_harga_jual,
      harga_beli = p_harga_beli,
      harga_servis = p_harga_servis
  WHERE uuid_produk = p_uuid_produk;

  UPDATE kondisi
  SET fisik = p_fisik,
      baut = p_baut,
      tutup_usb = p_tutup_usb,
      grip = p_grip,
      jamur_lensa = p_jamur_lensa,
      view_finder = p_view_finder,
```

```
      mounting = p_mounting,
      slot_memori = p_slot_memori,
      jamur_sensor = p_jamur_sensor,
      lcd = p_lcd,
      tombol = p_tombol,
      zoom_lensa = p_zoom_lensa,
      af_lensa = p_af_lensa,
      diafragma_lensa = p_diafragma_lensa,
      kalibrasi_fokus = p_kalibrasi_fokus,
      flash = p_flash,
      sound_mic = p_sound_mic,
      lain_lain = p_lain_lain
  WHERE uuid_kondisi = p_uuid_kondisi;

  UPDATE detail_produk
  SET serial_number = p_serial_number,
      serial_number_lens = p_serial_number_lens,
      stok_produk = p_stok_produk,
      deskripsi_produk = p_deskripsi_produk,
      gambar_produk = p_gambar_produk,
      status_produk = p_status_produk,
      kelengkapan = p_kelengkapan
  WHERE uuid_produk = p_uuid_produk;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk memperbarui informasi produk berdasarkan ID produk (uuid_produk).

ff. Stored Procedure get_produk

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `get_produk` ()
BEGIN
  SELECT
    p.*,
    dp.serial_number, dp.serial_number_lens, dp.stok_produk,
dp.deskripsi_produk, dp.status_produk, dp.kelengkapan,
    k.*
  FROM produk p
  JOIN detail_produk dp ON p.uuid_produk = dp.uuid_produk
  JOIN kondisi k ON dp.uuid_kondisi = k.uuid_kondisi;
END$$

CREATE DEFINER=`root`@`localhost` PROCEDURE `get_users` ()
BEGIN
  SELECT * FROM users;
```

```
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan seluruh data yang ada dalam tabel `produk`.

gg. Stored Procedure delete_produk

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `delete_produk` (
  IN `p_uuid_produk` VARCHAR(36)
)
BEGIN
  DECLARE v_uuid_kondisi VARCHAR(36);

  SELECT uuid_kondisi INTO v_uuid_kondisi
  FROM detail_produk
  WHERE uuid_produk = p_uuid_produk;

  DELETE FROM detail_produk WHERE uuid_produk = p_uuid_produk;
  DELETE FROM kondisi WHERE uuid_kondisi = v_uuid_kondisi;
  DELETE FROM produk WHERE uuid_produk = p_uuid_produk;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menghapus data produk berdasarkan `uuid_produk`.

hh. Stored Procedure create_pembelian

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `create_pembelian` (
  IN `p_uuid_beli` VARCHAR(36),
  IN `p_uuid_cust` VARCHAR(36),
  IN `p_uuid_perusahaan` VARCHAR(36),
  IN `p_uuid_user` VARCHAR(36),
  IN `p_tanggal` DATE,
  IN `p_kas` ENUM('transfer','cash'),
  IN `p_uuid_produk` VARCHAR(36),
  IN `p_serial_number` VARCHAR(20),
  IN `p_qty` INT,
  IN `p_uuid_kondisi` VARCHAR(36),
  IN `p_harga_tawaran` INT,
  IN `p_harga_terakhir` INT,
  IN `p_uuid_kelengkapan` VARCHAR(36),
  IN `p_status_pembelian` ENUM('deal','tidak')
)
```

```
BEGIN
  START TRANSACTION;

  INSERT INTO pembelian (
    uuid_beli, uuid_cust, uuid_perusahaan, uuid_user, tanggal, kas
  ) VALUES (
    p_uuid_beli, p_uuid_cust, p_uuid_perusahaan, p_uuid_user,
p_tanggal, p_kas
  );

  INSERT INTO detail_pembelian (
    uuid_beli, uuid_produk, serial_number, qty, uuid_kondisi,
    harga_tawaran, harga_terakhir, uuid_kelengkapan, status_pembelian
  ) VALUES (
    p_uuid_beli, p_uuid_produk, p_serial_number, p_qty,
p_uuid_kondisi,
    p_harga_tawaran, p_harga_terakhir, p_uuid_kelengkapan,
p_status_pembelian
  );

  COMMIT;
END$$

DELIMITER ;
```

Prosedur ini berfungsi untuk menambahkan data pembelian baru ke table pembelian.

ii. Stored Procedure update_pembelian

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `update_pembelian` (
  IN `p_uuid_beli` VARCHAR(36),
  IN `p_uuid_produk` VARCHAR(36),
  IN `p_uuid_cust` VARCHAR(36),
  IN `p_uuid_perusahaan` VARCHAR(36),
  IN `p_uuid_user` VARCHAR(36),
  IN `p_tanggal` DATE,
  IN `p_kas` ENUM('transfer','cash'),
  IN `p_serial_number` VARCHAR(20),
  IN `p_qty` INT,
  IN `p_uuid_kondisi` VARCHAR(36),
  IN `p_harga_tawaran` INT,
  IN `p_harga_terakhir` INT,
  IN `p_uuid_kelengkapan` VARCHAR(36),
  IN `p_status_pembelian` ENUM('deal','tidak')
)
BEGIN
  UPDATE pembelian
```

```
  SET uuid_cust = p_uuid_cust,
      uuid_perusahaan = p_uuid_perusahaan,
      uuid_user = p_uuid_user,
      tanggal = p_tanggal,
      kas = p_kas
  WHERE uuid_beli = p_uuid_beli;

  UPDATE detail_pembelian
  SET serial_number = p_serial_number,
      qty = p_qty,
      uuid_kondisi = p_uuid_kondisi,
      harga_tawaran = p_harga_tawaran,
      harga_terakhir = p_harga_terakhir,
      uuid_kelengkapan = p_uuid_kelengkapan,
      status_pembelian = p_status_pembelian
  WHERE uuid_beli = p_uuid_beli AND uuid_produk = p_uuid_produk;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk memperbarui informasi pembelian berdasarkan ID pembelian (`uuid_pembelian`).

jj. Stored Procedure get_pembelian

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `get_pembelian` (
  IN `p_uuid_beli` VARCHAR(36)
)
BEGIN
  SELECT
    pb.*, dp.*
  FROM pembelian pb
  JOIN detail_pembelian dp ON pb.uuid_beli = dp.uuid_beli
  WHERE pb.uuid_beli = p_uuid_beli;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan seluruh data yang ada dalam tabel `pembelian`.

kk. Stored Procedure delete_pembelian

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `delete_pembelian` (
  IN `p_uuid_beli` VARCHAR(36),
  IN `p_uuid_produk` VARCHAR(36)
)
BEGIN
  DELETE FROM detail_pembelian
  WHERE uuid_beli = p_uuid_beli AND uuid_produk = p_uuid_produk;

  IF NOT EXISTS (
    SELECT 1 FROM detail_pembelian WHERE uuid_beli = p_uuid_beli
  ) THEN
    DELETE FROM pembelian WHERE uuid_beli = p_uuid_beli;
  END IF;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menghapus data pembelian berdasarkan uuid_pembelian.

ll. Stored Procedure create_penjualan

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `create_penjualan` (
  IN `p_uuid_jual` VARCHAR(36),
  IN `p_uuid_cust` VARCHAR(36),
  IN `p_uuid_perusahaan` VARCHAR(36),
  IN `p_uuid_user` VARCHAR(36),
  IN `p_harga_total` INT,
  IN `p_diskon` DOUBLE,
  IN `p_kas` ENUM('transfer','cash'),
  IN `p_keterangan` VARCHAR(200),
  IN `p_tanggal` DATE,
  IN `p_uuid_produk` VARCHAR(36),
  IN `p_serial_number` VARCHAR(30),
  IN `p_qty` INT,
  IN `p_harga_depresiasi` DOUBLE
)
BEGIN
  INSERT INTO penjualan (
    uuid_jual, uuid_cust, uuid_perusahaan, uuid_user,
    harga_total, diskon, kas, keterangan, tanggal
  ) VALUES (
    p_uuid_jual, p_uuid_cust, p_uuid_perusahaan, p_uuid_user,
    p_harga_total, p_diskon, p_kas, p_keterangan, p_tanggal
```

```
  );

  INSERT INTO detail_penjualan (
    uuid_jual, uuid_produk, serial_number, qty, harga_depresiasi
  ) VALUES (
    p_uuid_jual, p_uuid_produk, p_serial_number, p_qty,
p_harga_depresiasi
  );

  COMMIT;
END$$

DELIMITER ;
```

Prosedur ini berfungsi untuk menambahkan data penjualan baru ke table penjualan.

mm. Stored Procedure update_penjualan

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `update_penjualan` (
  IN `p_uuid_jual` VARCHAR(36),
  IN `p_uuid_produk` VARCHAR(36),
  IN `p_uuid_cust` VARCHAR(36),
  IN `p_uuid_perusahaan` VARCHAR(36),
  IN `p_uuid_user` VARCHAR(36),
  IN `p_harga_total` INT,
  IN `p_diskon` DOUBLE,
  IN `p_kas` ENUM('transfer','cash'),
  IN `p_keterangan` VARCHAR(200),
  IN `p_tanggal` DATE,
  IN `p_serial_number` VARCHAR(30),
  IN `p_qty` INT,
  IN `p_harga_depresiasi` DOUBLE
)
BEGIN
  UPDATE penjualan
  SET uuid_cust = p_uuid_cust,
      uuid_perusahaan = p_uuid_perusahaan,
      uuid_user = p_uuid_user,
      harga_total = p_harga_total,
      diskon = p_diskon,
      kas = p_kas,
      keterangan = p_keterangan,
      tanggal = p_tanggal
  WHERE uuid_jual = p_uuid_jual;

  UPDATE detail_penjualan
  SET serial_number = p_serial_number,
      qty = p_qty,
```

```
      harga_depresiasi = p_harga_depresiasi
  WHERE uuid_jual = p_uuid_jual AND uuid_produk = p_uuid_produk;

  COMMIT;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk memperbarui informasi penjualan berdasarkan ID penjualan (`uuid_penjualan`).

nn. Stored Procedure get_penjualan

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `get_penjualan` (IN
`p_uuid_jual` VARCHAR(36))
BEGIN
    SELECT
        pj.*, dp.*
    FROM penjualan pj
    JOIN detail_penjualan dp ON pj.uuid_jual = dp.uuid_jual
    WHERE pj.uuid_jual = p_uuid_jual;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menampilkan seluruh data yang ada dalam tabel `penjualan`.

oo. Stored Procedure delete_penjualan

```
DELIMITER $$

CREATE DEFINER=`root`@`localhost` PROCEDURE `delete_penjualan` (IN
`p_uuid_jual` VARCHAR(36), IN `p_uuid_produk` VARCHAR(36))
BEGIN
    DELETE FROM detail_penjualan
    WHERE uuid_jual = p_uuid_jual AND uuid_produk = p_uuid_produk;

    IF NOT EXISTS (SELECT 1 FROM detail_penjualan WHERE uuid_jual =
p_uuid_jual) THEN
        DELETE FROM penjualan WHERE uuid_jual = p_uuid_jual;
    END IF;
END$$

DELIMITER ;
```

Prosedur ini digunakan untuk menghapus data kategori berdasarkan `uuid_penjualan`.

pp. Validasi Customer

```
SELECT COUNT(*) INTO v_customer_exists FROM customer WHERE uuid_cust
= p_uuid_cust;
IF v_customer_exists = 0 THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer tidak
ditemukan';
END IF;

IF NEW.no_telp NOT REGEXP '^[0-9]{10,13}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nomor telepon harus
berupa angka dan terdiri dari 10 hingga 13 digit';
END IF;

IF p_jenis_kelamin NOT IN ('L', 'P') THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Jenis kelamin harus L
atau P.';
END IF;
```

qq. Validasi perusahaan

```
SELECT COUNT(*) INTO v_perusahaan_exists FROM perusahaan_cabang WHERE
uuid_perusahaan = p_uuid_perusahaan;
IF v_perusahaan_exists = 0 THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Perusahaan tidak
ditemukan';
END IF;

SELECT COUNT(*) INTO v_nama_exists FROM perusahaan_cabang WHERE nama
= p_nama;
IF v_nama_exists > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nama perusahaan sudah
terdaftar';
END IF;

IF NEW.nomor_telepon NOT REGEXP '^[0-9]{10,13}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Nomor telepon harus
terdiri dari 10 sampai 13 digit angka';
END IF;
```

rr. Validasi User

```
    SELECT COUNT(*) INTO v_user_exists FROM users WHERE uuid_user =
p_uuid_user;
    IF v_user_exists = 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User tidak
ditemukan';
    END IF;
```

ss. Validasi Produk

```
SELECT COUNT(*) INTO v_produk_exists FROM produk WHERE uuid_produk =
p_uuid_produk;
IF v_produk_exists = 0 THEN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Produk tidak
ditemukan';
END IF;

SELECT COUNT(*) INTO v_sku_exists FROM produk WHERE sku = p_sku;
IF v_sku_exists > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'SKU sudah terdaftar';
END IF;

IF p_harga_jual <= 0 OR p_harga_beli <= 0 OR p_stok_produk <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Harga dan stok harus
lebih dari 0.';
END IF;

SELECT COUNT(*) INTO v_kondisi_exists FROM kondisi WHERE uuid_kondisi
= p_uuid_kondisi;
IF v_kondisi_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Kondisi tidak
ditemukan';
END IF;

IF p_harga_tawar <= 0 OR p_harga_akhir <= 0 OR p_qty <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Harga dan qty harus
lebih dari 0';
END IF;

IF p_serial_number IS NULL OR p_serial_number = '' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Serial number harus
diisi';
END IF;

SELECT stok_produk INTO v_stok FROM detail_produk WHERE uuid_produk =
NEW.uuid_produk;
IF v_stok < NEW.qty THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stok tidak mencukupi
untuk produk ini.';
END IF;
```

tt. Validasi Kategori

```
SELECT COUNT(*) INTO v_kategori_exists FROM kategori WHERE
uuid_kategori = p_uuid_kategori;
IF v_kategori_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Kategori tidak
ditemukan';
```

```
END IF;
```

uu. Validasi Pembelian

```
SELECT COUNT(*) INTO v_pembelian_exists FROM pembelian WHERE
uuid_beli = p_uuid_beli;
IF v_pembelian_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Pembelian tidak
ditemukan';
END IF;
```

vv. Validasi Penjualan

```
SELECT COUNT(*) INTO v_penjualan_exists FROM penjualan WHERE
uuid_jual = p_uuid_jual;
IF v_penjualan_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Penjualan tidak
ditemukan';
END IF;
IF p_harga_total <= 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Harga total harus
lebih dari 0';
END IF;
IF p_diskon < 0 OR p_diskon > 1 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Diskon harus antara 0
dan 1';
END IF;
```

ww. Validasi User

```
SELECT COUNT(*) INTO v_username_exists FROM users WHERE username =
p_username;
IF v_username_exists > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Username sudah
terdaftar';
END IF;

SELECT COUNT(*) INTO v_user_exists FROM users WHERE uuid_user =
p_uuid_user;
IF v_user_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User tidak
ditemukan';
END IF;

IF NEW.email NOT REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-
z]{2,}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Email yang dimasukkan
tidak valid. Pastikan format email benar.';
END IF;
```

```
IF LENGTH(NEW.password) < 8 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Password harus
minimal 8 karakter.';
END IF;
```

xx. Validasi Karyawan

```
SELECT COUNT(*) INTO v_karyawan_exists FROM karyawan WHERE
uuid_karyawan = p_uuid_karyawan;
IF v_karyawan_exists = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Karyawan tidak
ditemukan';
END IF;

IF NEW.nik NOT REGEXP '^[0-9]{16}$' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'NIK harus terdiri
dari 16 digit angka';
END IF;
```

yy. Trigger after_insert_pembelian

```
DELIMITER $$
CREATE TRIGGER after_insert_pembelian
AFTER INSERT ON detail_pembelian
FOR EACH ROW
BEGIN
    UPDATE detail_produk
    SET stok_produk = stok_produk + NEW.qty
    WHERE uuid_produk = NEW.uuid_produk;
END$$
DELIMITER ;
```

zz. Trigger after_insert_penjualan

```
DELIMITER $$
CREATE TRIGGER after_insert_penjualan
AFTER INSERT ON detail_penjualan
FOR EACH ROW
BEGIN
    DECLARE v_stok INT;

    -- Ambil stok saat ini
    SELECT stok_produk INTO v_stok
    FROM detail_produk
    WHERE uuid_produk = NEW.uuid_produk;

    -- Validasi stok mencukupi
    IF v_stok < NEW.qty THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stok tidak
mencukupi untuk produk ini.';
    END IF;

    -- Kurangi stok
    UPDATE detail_produk
    SET stok_produk = stok_produk - NEW.qty
    WHERE uuid_produk = NEW.uuid_produk;
END$$
DELIMITER ;
```

VI. Hasil Query

a. Tabel Users

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP coc

uuid_user	username	password	email	role

Query results operations

b. Tabel produk

```
SELECT * FROM `produk`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_produk	kode_sku	uuid_kategori	nama_produk	harga_jual	harga_beli	harga_servis

Query results operations

c. Tabel penjualan

```
SELECT * FROM `penjualan`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_jual	uuid_cust	uuid_perusahaan	uuid_user	harga_total	diskon	kas	keterangan	tanggal

Query results operations

d. Tabel pembelian

```
SELECT * FROM `pembelian`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_beli	uuid_cust	uuid_perusahaan	uuid_user	tanggal	kas

Query results operations

e. Tabel kondisi

```
SELECT * FROM `kondisi`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_kondisi	fisik	baut	tutup_usb	grip	jamur_lensa	view_finder	mounting	slot_memori	jamur_sensor	lcd	tombol	zoom_lensa	af_lensa	diafragma_lensa	kalibrasi_fokus	flash	sound_mic	lain_lain

f. Tabel kategori

db_dinoyocamera
- New
- kategori
- users
- dinoyo_camf

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [

uuid_kategori	nama_kategori

Query results operations

g. Tabel karyawan

```
SELECT * FROM `karyawan`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_karyawan	uuid_user	nama_lengkap	nik	jabatan	gaji	tanggal_masuk	tanggal_keluar	status	nomor_telepon	alamat

Query results operations

h. Tabel perusahaan cabang

```
SELECT * FROM `perusahaan_cabang`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_perusahaan	nama	alamat	nomor_telepon

i. Tabel detail produk

```
SELECT * FROM `detail_produk`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_produk	serial_number	serial_number_lens	stok_produk	deskripsi_produk	gambar_produk	status_produk	kelengkapan

j. **Tabel detail penjualan**

```
SELECT * FROM `detail_penjualan`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_jual	uuid_produk	serial_number	qty	harga_depresiasi

Query results operations

k. **Tabel detail pembelian**

```
SELECT * FROM `detail_pembelian`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_beli	uuid_produk	serial_number	uuid_kondisi	harga_tawaran	harga_terakhir	status_pembelian

Query results operations

l. **Tabel customer**

```
SELECT * FROM `customer`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

uuid_cust	nama	identitas	no_telp	alamat	jenis_kelamin	referensi	keterangan

Query results operations

m. **Stored Procedure create_users call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL create_user(
    @p_username,
    @p_password,
    @p_email,
    @p_role
);
```

n. **Stored Procedure get_users call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL get_users();
```

o. **Stored Procedure update_users call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL update_user(
    @p_uuid_user,
    @p_username,
    @p_password,
    @p_email,
    @p_role
);
```

p. **Stored Procedure delete_users call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL delete_user(@p_uuid_user);
```

q. **Stored Procedure create_karyawan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL create_karyawan(
    @p_nama_karyawan,
    @p_email,
    @p_nomor_hp,
    @p_jabatan,
    @p_id_perusahaan
);
```

r. **Stored Procedure create_perusahaan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL create_perusahaan(
    @p_nama_perusahaan,
    @p_alamat,
    @p_telepon,
    @p_email
)
```

s. **Stored Procedure delete_karyawan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL delete_karyawan(@p_uuid_karyawan);
```

t. **Stored Procedure delete_perusahaan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL delete_perusahaan(@p_uuid_perusahaan);
```

u. **Stored Procedure get_karyawan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL get_karyawan();
```

v. **Stored Procedure get_perusahaan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL get_perusahaan();
```

w. **Stored Procedure update_karyawan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL update_karyawan(
    @p_uuid_karyawan,
    @p_nama_karyawan,
    @p_email,
    @p_nomor_hp,
    @p_jabatan,
    @p_id_perusahaan
);
```

x. **Stored Procedure update_karyawan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL update_karyawan(
    @p_uuid_karyawan,
    @p_nama_karyawan,
    @p_email,
    @p_nomor_hp,
    @p_jabatan,
    @p_id_perusahaan
);
```

y. **Stored Procedure update_perusahaan call query**

Run SQL query/queries on database db_dinoyocamera:

```
CALL update_perusahaan(
    @p_uuid_perusahaan,
    @p_nama_perusahaan,
    @p_alamat,
    @p_telepon,
    @p_email
);

```

VII. Kesimpulan

Pengembangan Sistem Manajemen *Database* Terintegrasi ini telah memberikan kesempatan berharga bagi mahasiswa untuk mengimplementasikan konsep basis data relasional dalam lingkungan nyata. Melalui proyek ini, mahasiswa tidak hanya berhasil menerapkan pengetahuan teoretis tentang normalisasi data, perancangan relasi antar tabel, dan pengembangan *stored procedure*, tetapi juga mengembangkan kemampuan analisis kebutuhan pengguna dan pemecahan masalah bisnis secara praktis. Pengalaman kolaborasi langsung dengan mitra UMKM telah memperkaya pemahaman mahasiswa tentang pentingnya sistem *database* yang terstruktur sebagai landasan digitalisasi usaha, sekaligus membangun keterampilan *soft skill* dalam komunikasi dan kerja sama tim yang esensial untuk karir profesional di bidang teknologi informasi.

Secara lebih luas, proyek ini merefleksikan peran krusial akademisi dalam mendukung transformasi digital UMKM di Indonesia. Digitalisasi melalui implementasi sistem manajemen *database* relasional menawarkan solusi yang terjangkau namun berdampak signifikan bagi sektor UMKM dalam meningkatkan efisiensi operasional, transparansi transaksi, dan pengambilan keputusan berbasis data. Dengan semakin banyak mahasiswa yang terlibat dalam pengembangan solusi teknologi praktis seperti ini, tercipta ekosistem kolaborasi antara dunia pendidikan dan sektor usaha yang dapat mempercepat adopsi teknologi di kalangan UMKM Indonesia. Pada akhirnya, inisiatif semacam ini berkontribusi pada penguatan fondasi ekonomi digital nasional dan peningkatan daya saing UMKM dalam menghadapi era revolusi industri 4.0.